

Toll Tax Management System

1. Team Details

- Kanishk Vikram Singh [SAP ID: 500121743]
 - Sumit Kumar [SAP ID: 500119644]
-

2. Topic

Toll Tax Management System

3. Abstract

This project implements a console-based Toll Tax Management System using Java, aimed at streamlining the toll collection process for different categories of vehicles. It incorporates object-oriented programming principles such as inheritance, abstraction, and polymorphism to represent vehicle types including LMVs, LCVs, and Heavy Vehicles. The system supports toll calculation, transaction record keeping, revenue generation by vehicle category, and automatic receipt generation using file handling operations. It serves as a comprehensive demonstration of Java OOP concepts integrated with file management.

4. Main Functionalities

- **Categorization of Vehicles:** The system differentiates vehicles into three major categories—Light Motor Vehicles (LMV), Light Commercial Vehicles (LCV), and Heavy Vehicles—each with its own toll rate logic.
 - **Dynamic Toll Calculation:** Toll charges are dynamically computed based on the type of vehicle selected. This is handled through method overriding in each derived vehicle class.
 - **Persistent Record Maintenance Using File Handling:** All toll transactions are saved to a file using Java's file handling capabilities, ensuring data persistence even after the program is closed.
 - **Display of All Transaction Records:** Users can view a complete list of toll transactions, including vehicle details and associated toll charges.
 - **Deletion of Specific Vehicle Toll Records:** The system allows users to delete toll records of specific vehicles using their unique vehicle numbers. The corresponding file is updated accordingly.
 - **Revenue Generation Reports by Vehicle Category:** The total toll collected is summarized by vehicle category (LMV, LCV, Heavy Vehicle), offering insights into toll collection trends.
 - **Automatic Receipt Generation for Toll Payment:** Upon toll collection, a formatted receipt containing vehicle and toll information is automatically generated and saved as a separate file.
 - **Menu-Driven Interface for Seamless User Interaction:** A simple and intuitive text-based menu guides users through all the functionalities, making the system user-friendly and easy to navigate.
-

5. Complete Code (Text Format)

```
1. import java.io.BufferedReader;
2. import java.io.FileReader;
3. import java.io.FileWriter;
4. import java.io.IOException;
5. import java.util.InputMismatchException;
6. import java.util.Scanner;
7.
8. // This is my Base class for Vehicles
9. abstract class Vehicle {
10.
11.     protected String vehicleNumber;
12.     protected String vehicleType;
13.
14.     public Vehicle(String vehicleNumber, String vehicleType) {
15.         this.vehicleNumber = vehicleNumber;
16.         this.vehicleType = vehicleType;
17.     }
18.
19.     public abstract double calculateToll();
20.
21.     public void displayInfo() {
22.         System.out.println("Vehicle Number: " + vehicleNumber);
23.         System.out.println("Vehicle Type: " + vehicleType);
24.         System.out.println("Toll Amount: " + calculateToll());
25.     }
26.
27.     public String toFileString() {
28.         return vehicleNumber + "," + vehicleType + "," + calculateToll();
29.     }
30. }
31.
32. // Derived classes for specific vehicle types
33. class LMV extends Vehicle {
34.
```

```
35. private String LMVtype;
36.
37. public LMV(String vehicleNumber, String LMVtype) {
38.     super(vehicleNumber, "Light Motor Vehicle");
39.     this.LMVtype = LMVtype;
40. }
41.
42. @Override
43. public double calculateToll() {
44.     switch (LMVtype.toLowerCase()) {
45.         case "suv":
46.             return 70.0;
47.         case "sedan":
48.             return 60.0;
49.         case "hatchback":
50.             return 50.0;
51.         case "jeep":
52.             return 40.0;
53.         default:
54.             return 50.0;
55.     }
56. }
57.
58. @Override
59. public void displayInfo() {
60.     super.displayInfo();
61.     System.out.println("Car Type: " + LMVtype);
62. }
63.
64. @Override
65. public String toFileString() {
66.     return vehicleNumber + "," + vehicleType + "," + LMVtype + "," + calculateToll();
67. }
68. }
69.
70. class LCV extends Vehicle {
71.
72.     private String LCVtype;
73.
74.     public LCV(String vehicleNumber, String LCVtype) {
75.         super(vehicleNumber, "Light Commercial Vehicle");
```

```
76.     this.LCVtype = LCVtype;
77. }
78.
79. public double calculateToll() {
80.     switch (LCVtype.toLowerCase()) {
81.         case "minibus":
82.             return 100.0;
83.         case "goods vehicle":
84.             return 120.0;
85.         default:
86.             return 100.0;
87.     }
88. }
89.
90. public void displayInfo() {
91.     super.displayInfo();
92.     System.out.println("LCV type: " + LCVtype);
93. }
94.
95. public String toFileString() {
96.     return vehicleNumber + "," + vehicleType + "," + LCVtype + "," + calculateToll();
97. }
98. }
99.
100. class HeavyVehicle extends Vehicle {
101.
102.     private String HeavyVehicleType;
103.
104.     public HeavyVehicle(String vehicleNumber, String HeavyVehicleType) {
105.         super(vehicleNumber, "Heavy Vehicle");
106.         this.HeavyVehicleType = HeavyVehicleType;
107.     }
108.
109.     public double calculateToll() {
110.         switch (HeavyVehicleType.toLowerCase()) {
111.             case "bus":
112.                 return 150.0;
113.             case "truck":
114.                 return 170.0;
115.             case "construction vehicle":
116.                 return 200.0;
```

```
117.     default:
118.         return 150.0;
119.     }
120. }
121.
122. public void displayInfo() {
123.     super.displayInfo();
124.     System.out.println("Heavy Vehicle Type: " + HeavyVehicleType);
125. }
126.
127. public String toFileString() {
128.     return vehicleNumber + "," + vehicleType + "," + HeavyVehicleType + "," + calculateToll();
129. }
130. }
131.
132. // TollBooth class to handle toll collection
133. class TollBooth {
134.
135.     private static final String FILE_NAME = "toll_records.txt";
136.     private Vehicle[] vehicleRecords = new Vehicle[100]; // Used Fixed-size array instead of ArrayList
137.     private int count = 0;
138.
139.     public TollBooth() {
140.         loadRecordsFromFile();
141.     }
142.
143.     public void deleteRecord(String vehicleNumber) {
144.         boolean found = false;
145.         int index = -1;
146.
147.         // Find the record to delete
148.         for (int i = 0; i < count; i++) {
149.             if (vehicleRecords[i].vehicleNumber.equals(vehicleNumber)) {
150.                 found = true;
151.                 index = i;
152.                 break;
153.             }
154.         }
155.
156.         if (found) {
157.             // Shift elements left to remove the deleted record
```

```

158.     for (int j = index; j < count - 1; j++) {
159.         vehicleRecords[j] = vehicleRecords[j + 1];
160.     }
161.     count--; // Reduce count after deletion
162.
163.     // Update file after deletion
164.     try (FileWriter fw = new FileWriter(FILE_NAME)) {
165.         for (int i = 0; i < count; i++) {
166.             fw.write(vehicleRecords[i].toFileString() + "\n");
167.         }
168.         System.out.println("Record deleted successfully.");
169.     } catch (IOException e) {
170.         System.out.println("Error updating the file: " + e.getMessage());
171.     }
172. } else {
173.     System.out.println("Vehicle number not found.");
174. }
175. }
176.
177. public void collectToll(Vehicle vehicle) {
178.     if (count < vehicleRecords.length) {
179.         vehicleRecords[count++] = vehicle;
180.         System.out.println("Processing Toll for Vehicle: " + vehicle.vehicleNumber);
181.         vehicle.displayInfo();
182.         System.out.println("Toll Paid Successfully!\n");
183.         writeRecordToFile(vehicle);
184.         generateReceipt(vehicle);
185.     } else {
186.         System.out.println("Toll booth storage is full!");
187.     }
188. }
189.
190. private void generateReceipt(Vehicle vehicle) {
191.     String receiptFileName = "Receipt_" + vehicle.vehicleNumber + ".txt";
192.     try (FileWriter fw = new FileWriter(receiptFileName)) {
193.         fw.write("----- Toll Payment Receipt ----- \n");
194.         fw.write("Vehicle Number: " + vehicle.vehicleNumber + " \n");
195.         fw.write("Vehicle Type: " + vehicle.vehicleType + " \n");
196.         fw.write("Toll Amount Paid: ₹" + vehicle.calculateToll() + " \n");
197.         fw.write("Payment Status: SUCCESS \n");
198.         fw.write("----- \n");

```

```
199.     System.out.println("Receipt Generated: " + receiptFileName);
200. } catch (IOException e) {
201.     System.out.println("Error generating receipt: " + e.getMessage());
202. }
203. }
204.
205. public void displayRecords() {
206.     System.out.println("--- Toll Collection Records ---");
207.     double totalRevenue = 0.0;
208.
209.     for (int i = 0; i < count; i++) {
210.         totalRevenue += vehicleRecords[i].calculateToll();
211.         vehicleRecords[i].displayInfo();
212.         System.out.println();
213.     }
214.     System.out.println("Total Revenue Collected: ₹" + totalRevenue);
215. }
216.
217. private void writeRecordToFile(Vehicle vehicle) {
218.     try (FileWriter fw = new FileWriter(FILE_NAME, true)) {
219.         fw.write(vehicle.toFileString() + "\n");
220.     } catch (IOException e) {
221.         System.out.println("Error writing to file: " + e.getMessage());
222.     }
223. }
224.
225. private void loadRecordsFromFile() {
226.     try (BufferedReader br = new BufferedReader(new FileReader(FILE_NAME))) {
227.         String line;
228.         while ((line = br.readLine()) != null && count < vehicleRecords.length) {
229.             String[] parts = line.split(",");
230.             Vehicle vehicle = null;
231.             if (parts[1].equals("Light Motor Vehicle")) {
232.                 vehicle = new LMV(parts[0], parts[2]);
233.             } else if (parts[1].equals("Light Commercial Vehicle")) {
234.                 vehicle = new LCV(parts[0], parts[2]);
235.             } else if (parts[1].equals("Heavy Vehicle")) {
236.                 vehicle = new HeavyVehicle(parts[0], parts[2]);
237.             }
238.             if (vehicle != null) {
239.                 vehicleRecords[count++] = vehicle;
```



```

240.     }
241. }
242. } catch (IOException e) {
243.     System.out.println("No previous records found.");
244. }
245. }
246.
247. public void writeRevenueByVehicleType() {
248.     double LMVTotal = 0, LCVTotal = 0, HVTTotal = 0;
249.
250.     try (FileWriter lmvwrite = new FileWriter("LMV_Revenue.txt"); FileWriter lcvwrite = new FileWriter("LCV_Revenue.txt"); FileWriter hvwrite = new
FileWriter("HV_Revenue.txt")) {
251.
252.         for (int i = 0; i < count; i++) {
253.             Vehicle v = vehicleRecords[i];
254.             double toll = v.calculateToll();
255.
256.             if (v instanceof LMV) {
257.                 LMVTotal += toll;
258.                 lmvwrite.write(v.toFileString() + "\n");
259.             } else if (v instanceof LCV) {
260.                 LCVTotal += toll;
261.                 lcvwrite.write(v.toFileString() + "\n");
262.             } else if (v instanceof HeavyVehicle) {
263.                 HVTTotal += toll;
264.                 hvwrite.write(v.toFileString() + "\n");
265.             }
266.         }
267.
268.         lmvwrite.write("Total LMV Revenue: ₹" + LMVTotal + "\n");
269.         lcvwrite.write("Total LCV Revenue: ₹" + LCVTotal + "\n");
270.         hvwrite.write("Total Heavy Vehicle Revenue: ₹" + HVTTotal + "\n");
271.     } catch (IOException e) {
272.         System.out.println("Error writing revenue Files: " + e.getMessage());
273.     }
274.
275.     System.out.println("Revenue reports Generated Successfully !");
276. }
277. }
278.
279. // Main system controller

```

```

280. public class TollSystem {
281.
282.     public static void main(String[] args) {
283.         Scanner scanner = new Scanner(System.in);
284.         TollBooth tollBooth = new TollBooth();
285.
286.         while (true) {
287.             System.out.println("\n----- Welcome to my Toll Tax Management System-----");
288.             System.out.println("1. Pay Toll");
289.             System.out.println("2. Display Records");
290.             System.out.println("3. Generate Revenue Report");
291.             System.out.println("4. Delete Record");
292.             System.out.println("5. Exit");
293.             System.out.print("Choose an option: ");
294.
295.             try {
296.                 int choice = scanner.nextInt();
297.                 scanner.nextLine(); // Consume newline character
298.
299.                 if (choice == 1) {
300.                     System.out.print("Enter Vehicle Number: ");
301.                     String vehicleNumber = scanner.nextLine();
302.                     System.out.println("Select Vehicle Type: 1. LMV 2. LCV 3. Heavy Vehicle");
303.                     int type = scanner.nextInt();
304.                     scanner.nextLine();
305.
306.                     Vehicle vehicle = null;
307.                     if (type == 1) {
308.                         System.out.println("Select LMV Type: 1. SUV 2. Sedan 3. Hatchback 4. Jeep");
309.                         int LMVchoice = scanner.nextInt();
310.                         scanner.nextLine();
311.                         String LMVtype = (LMVchoice == 1) ? "SUV" : (LMVchoice == 2) ? "Sedan" : (LMVchoice == 3) ? "Hatchback" : "Jeep";
312.                         vehicle = new LMV(vehicleNumber, LMVtype);
313.                     } else if (type == 2) {
314.                         System.out.println("Select LCV Type: 1. Minibus 2. Goods Vehicle");
315.                         int LCVchoice = scanner.nextInt();
316.                         scanner.nextLine();
317.                         String LCVtype = (LCVchoice == 1) ? "Minibus" : "Goods Vehicle";
318.                         vehicle = new LCV(vehicleNumber, LCVtype);
319.                     } else if (type == 3) {
320.                         System.out.println("Select Heavy Vehicle Type: 1. Bus 2. Truck 3. Construction Vehicle");

```

```
321.         int HVchoice = scanner.nextInt();
322.         scanner.nextLine();
323.         String HVtype = (HVchoice == 1) ? "Bus" : (HVchoice == 2) ? "Truck" : "Construction Vehicle";
324.         vehicle = new HeavyVehicle(vehicleNumber, HVtype);
325.     }
326.     tollBooth.collectToll(vehicle);
327. } else if (choice == 2) {
328.     tollBooth.displayRecords();
329. } else if (choice == 3) {
330.     tollBooth.writeRevenueByVehicleType();
331. } else if (choice == 4) {
332.     System.out.print("Enter Vehicle Number to Delete: ");
333.     String vehicleNumber = scanner.nextLine();
334.     tollBooth.deleteRecord(vehicleNumber);
335. } else if (choice == 5) {
336.     break;
337. } else {
338.     System.out.println("Invalid choice");
339. }
340. } catch (InputMismatchException e) {
341.     System.out.println("Enter a number please !");
342.     scanner.nextLine();
343. }
344. }
345. scanner.close();
346. }
347. }
348.
```

6. Screenshots with Explanations

Parent Class Vehicle

```
// This is my Base class for Vehicles
abstract class Vehicle {

    protected String vehicleNumber;
    protected String vehicleType;

    public Vehicle(String vehicleNumber, String vehicleType) {
        this.vehicleNumber = vehicleNumber;
        this.vehicleType = vehicleType;
    }

    public abstract double calculateToll();

    public void displayInfo() {
        System.out.println("Vehicle Number: " + vehicleNumber);
        System.out.println("Vehicle Type: " + vehicleType);
        System.out.println("Toll Amount: " + calculateToll());
    }

    public String toFileString() {
        return vehicleNumber + "," + vehicleType + "," + calculateToll();
    }
}
```

Figure 1

This is the abstract base class representing a generic vehicle in the Toll Tax Management System. It defines the common structure and behaviors shared by all vehicle types.

Constructor: Initializes the vehicle number and vehicle type, which are common across all vehicle categories.

calculateToll() (Abstract Method): Declared as abstract to enforce that all subclasses must implement their specific toll calculation logic based on vehicle category.

displayInfo(): Displays the vehicle number, type, and the toll amount on the console.

toFileString(): Returns a comma-separated string of vehicle details and the calculated toll, which is useful for writing data to a file.

Class: LMV (Light Motor Vehicle – Derived Class)

```
// Derived classes for specific vehicle types
class LMV extends Vehicle {

    private String LMVtype;

    public LMV(String vehicleNumber, String LMVtype) {
        super(vehicleNumber, "Light Motor Vehicle");
        this.LMVtype = LMVtype;
    }

    @Override
    public double calculateToll() {
        switch (LMVtype.toLowerCase()) {
            case "suv":
                return 70.0;
            case "sedan":
                return 60.0;
            case "hatchback":
                return 50.0;
            case "jeep":
                return 40.0;
            default:
                return 50.0;
        }
    }

    @Override
    public void displayInfo() {
        super.displayInfo();
        System.out.println("Car Type: " + LMVtype);
    }

    @Override
    public String toFileString() {
        return vehicleNumber + "," + vehicleType + "," + LMVtype + "," + calculateToll();
    }
}
```

Figure 2

This class represents **Light Motor Vehicles (LMVs)** such as sedans, hatchbacks, SUVs, and jeeps. It extends the abstract base class Vehicle and provides specific implementations for toll calculation.

Constructor: Initializes the vehicle number and LMV subtype (e.g., SUV, sedan) and passes "Light Motor Vehicle" as the general type to the base class.

calculateToll(): Overrides the abstract method to compute toll charges dynamically based on the LMV subtype using a switch statement. For instance:

SUV → ₹70.0

Sedan → ₹60.0

Hatchback → ₹50.0

Jeep → ₹40.0

displayInfo(): Overrides the base method to include the specific LMV type along with other vehicle details.

toFileString(): Returns a string with all relevant details (vehicle number, type, subtype, and toll) for saving to the records file.

Class: LCV (Light Commercial Vehicle – Derived Class)

```
class LCV extends Vehicle {  
  
    private String LCVtype;  
  
    public LCV(String vehicleNumber, String LCVtype) {  
        super(vehicleNumber, "Light Commercial Vehicle");  
        this.LCVtype = LCVtype;  
    }  
  
    public double calculateToll() {  
        switch (LCVtype.toLowerCase()) {  
            case "minibus":  
                return 100.0;  
            case "goods vehicle":  
                return 120.0;  
            default:  
                return 100.0;  
        }  
    }  
  
    public void displayInfo() {  
        super.displayInfo();  
        System.out.println("LCV type: " + LCVtype);  
    }  
  
    public String toFileString() {  
        return vehicleNumber + "," + vehicleType + "," + LCVtype + "," + calculateToll();  
    }  
}
```

The LCV class extends the abstract Vehicle class to represent **Light Commercial Vehicles** like minibuses and goods carriers.

Constructor: Accepts the vehicle number and specific LCV type, and assigns "Light Commercial Vehicle" as the general type to the superclass.

calculateToll(): Implements toll logic based on the LCV subtype. The toll is calculated as:

Minibus → ₹100.0

Goods Vehicle → ₹120.0

Any other unspecified type → ₹100.0 (default)

displayInfo(): Adds the LCV subtype detail along with the standard vehicle information.

toFileString(): Returns a string format with all relevant LCV data for writing to the records file.

This class illustrates **polymorphism** through method overriding and provides tailored toll rates for different commercial light vehicles.

Class: HeavyVehicle (Heavy Vehicle – Derived Class)

```
class HeavyVehicle extends Vehicle {  
  
    private String HeavyVehicleType;  
  
    public HeavyVehicle(String vehicleNumber, String HeavyVehicleType) {  
        super(vehicleNumber, "Heavy Vehicle");  
        this.HeavyVehicleType = HeavyVehicleType;  
    }  
  
    public double calculateToll() {  
        switch (HeavyVehicleType.toLowerCase()) {  
            case "bus":  
                return 150.0;  
            case "truck":  
                return 170.0;  
            case "construction vehicle":  
                return 200.0;  
            default:  
                return 150.0;  
        }  
    }  
  
    public void displayInfo() {  
        super.displayInfo();  
        System.out.println("Heavy Vehicle Type: " + HeavyVehicleType);  
    }  
  
    public String toFileString() {  
        return vehicleNumber + "," + vehicleType + "," + HeavyVehicleType + "," + calculateToll();  
    }  
}
```

The HeavyVehicle class inherits from the Vehicle base class and is used to represent large and heavy-duty vehicles such as buses, trucks, and construction equipment.

Constructor: Initializes the vehicle number and a specific heavy vehicle type, and sets the general type as "Heavy Vehicle" via the superclass constructor.

calculateToll(): Overrides the abstract method to compute tolls based on the subtype:

Bus → ₹150.0

Truck → ₹170.0

Construction Vehicle → ₹200.0

Any unlisted type → ₹150.0 (default)

displayInfo(): Displays all standard vehicle info and adds the specific heavy vehicle subtype.

toFileString(): Returns a CSV-formatted string for writing this vehicle's data to the transaction file.

This class further demonstrates the use of **inheritance and method overriding**, enabling specific toll logic for heavy transport vehicles.

Function: deleteRecord(String vehicleNumber)

```
class TollBooth {  
  
    private static final String FILE_NAME = "toll_records.txt";  
    private Vehicle[] vehicleRecords = new Vehicle[100]; // Used Fixed-size array instead of ArrayList  
    private int count = 0;  
  
    public TollBooth() {  
        loadRecordsFromFile();  
    }  
  
    public void deleteRecord(String vehicleNumber) {  
        boolean found = false;  
        int index = -1;  
  
        // Find the record to delete  
        for (int i = 0; i < count; i++) {  
            if (vehicleRecords[i].vehicleNumber.equals(vehicleNumber)) {  
                found = true;  
                index = i;  
                break;  
            }  
        }  
  
        if (found) {  
            // Shift elements left to remove the deleted record  
            for (int j = index; j < count - 1; j++) {  
                vehicleRecords[j] = vehicleRecords[j + 1];  
            }  
            count--; // Reduce count after deletion  
  
            // Update file after deletion  
            try (FileWriter fw = new FileWriter(FILE_NAME)) {  
                for (int i = 0; i < count; i++) {  
                    fw.write(vehicleRecords[i].toFileString() + "\n");  
                }  
                System.out.println("Record deleted successfully.");  
            } catch (IOException e) {  
                System.out.println("Error updating the file: " + e.getMessage());  
            }  
        } else {  
            System.out.println("Vehicle number not found.");  
        }  
    }  
}
```

```
----- Welcome to my Toll Tax Management System-----
1. Pay Toll
2. Display Records
3. Generate RHevenue Report
4. Delete Record
5. Exit
Choose an option: 4
Enter Vehicle Number to Delete: UP32EP0716
Record deleted successfully.

----- Welcome to my Toll Tax Management System-----
1. Pay Toll
2. Display Records
3. Generate RHevenue Report
4. Delete Record
5. Exit
Choose an option: 5
```

This method enables deletion of a specific toll record using the vehicle number, from both memory and file storage.

- **Search Operation:** Iterates through the vehicleRecords array to find the matching vehicle number.
- **Shifting Mechanism:** If found, it shifts all elements after the found index one position to the left, effectively removing the targeted record.
- **Record Count Management:** Decreases the count variable to reflect the new number of records.
- **File Update:** Rewrites all remaining records back to the file using FileWriter, ensuring that the deleted record is permanently removed from storage.
- **Error Handling:** Catches and reports any IOException that might occur during file writing.
- **Feedback:** Prints confirmation messages for both successful deletion and for cases where the vehicle number was not found.

This function demonstrates file handling, in-place deletion logic, and robust error control, forming a key part of the system's record maintenance capabilities.

Function: collectToll(Vehicle vehicle)

```
public void collectToll(Vehicle vehicle) {  
    if (count < vehicleRecords.length) {  
        vehicleRecords[count++] = vehicle;  
        System.out.println("Processing Toll for Vehicle: " + vehicle.vehicleNumber);  
        vehicle.displayInfo();  
        System.out.println("Toll Paid Successfully!\n");  
        writeRecordToFile(vehicle);  
        generateReceipt(vehicle);  
    } else {  
        System.out.println("Toll booth storage is full!");  
    }  
}
```

```
----- Welcome to my Toll Tax Management System-----  
1. Pay Toll  
2. Display Records  
3. Generate Revenue Report  
4. Delete Record  
5. Exit  
Choose an option: 1  
Enter Vehicle Number: UP32EP0716  
Select Vehicle Type: 1. LMV 2. LCV 3. Heavy Vehicle  
1  
Select LMV Type: 1. SUV 2. Sedan 3. Hatchback 4. Jeep  
3  
Processing Toll for Vehicle: UP32EP0716  
Vehicle Number: UP32EP0716  
Vehicle Type: Light Motor Vehicle  
Toll Amount: 50.0  
Car Type: Hatchback  
Toll Paid Successfully!  
  
Receipt Generated: Receipt_UP32EP0716.txt
```

This function is responsible for processing the toll transaction of an incoming vehicle and handling all related operations.

- **Record Storage:** Adds the vehicle object to the vehicleRecords array and increments the count, keeping track of total entries.
- **Vehicle Processing Feedback:** Displays real-time feedback in the console indicating the vehicle being processed.
- **Display Details:** Calls vehicle.displayInfo() to show all relevant vehicle and toll information to the user.
- **Toll Confirmation:** Prints a success message confirming the toll has been paid.
- **File Logging:** Uses the writeRecordToFile(vehicle) method to store the transaction in a persistent file.
- **Receipt Generation:** Automatically generates a toll receipt using the generateReceipt(vehicle) function for recordkeeping and user reference.
- **Storage Limit Handling:** Checks for array overflow and informs the user if the toll booth storage is full.

This method acts as the heart of the toll system by integrating object storage, user interaction, file handling, and receipt automation.

Function: generateReceipt(Vehicle vehicle)

```
private void generateReceipt(Vehicle vehicle) {  
    String receiptFileName = "Receipt_" + vehicle.vehicleNumber + ".txt";  
    try (FileWriter fw = new FileWriter(receiptFileName)) {  
        fw.write("----- Toll Payment Receipt -----\\n");  
        fw.write("Vehicle Number: " + vehicle.vehicleNumber + "\\n");  
        fw.write("Vehicle Type: " + vehicle.vehicleType + "\\n");  
        fw.write("Toll Amount Paid: ₹" + vehicle.calculateToll() + "\\n");  
        fw.write("Payment Status: SUCCESS\\n");  
        fw.write("-----\\n");  
        System.out.println("Receipt Generated: " + receiptFileName);  
    } catch (IOException e) {  
        System.out.println("Error generating receipt: " + e.getMessage());  
    }  
}
```

≡ Receipt_UP32EP0716.txt

```
1  ----- Toll Payment Receipt -----  
2  Vehicle Number: UP32EP0716  
3  Vehicle Type: Light Motor Vehicle  
4  Toll Amount Paid: ₹50.0  
5  Payment Status: SUCCESS  
6  -----  
7
```

This function is responsible for generating a formatted text receipt for each toll payment made.

- **Dynamic File Naming:** Creates a unique receipt file for each vehicle using its vehicle number (e.g., Receipt_UP32XX1234.txt), ensuring no overwriting occurs.
- **Receipt Formatting:** Writes structured receipt content that includes:
 - Vehicle number
 - Vehicle type
 - Toll amount paid
 - Payment status
- **File Writing:** Uses FileWriter within a try-with-resources block for safe and automatic resource management.
- **Success Message:** Prints a success confirmation in the console indicating the receipt has been created.
- **Error Handling:** Catches and displays any IOException that may occur during file writing.

This function automates and standardizes the receipt creation process, offering users a reliable and printable proof of toll payment.

Functions: displayRecords() and writeRecordToFile(Vehicle vehicle)

```
public void displayRecords() {
    System.out.println("--- Toll Collection Records ---");
    double totalRevenue = 0.0;

    for (int i = 0; i < count; i++) {
        totalRevenue += vehicleRecords[i].calculateToll();
        vehicleRecords[i].displayInfo();
        System.out.println();
    }
    System.out.println("Total Revenue Collected: ₹" + totalRevenue);
}

private void writeRecordToFile(Vehicle vehicle) {
    try (FileWriter fw = new FileWriter(FILE_NAME, true)) {
        fw.write(vehicle.toFileString() + "\n");
    } catch (IOException e) {
        System.out.println("Error writing to file: " + e.getMessage());
    }
}
```

```
----- Welcome to my Toll Tax Management System-----
1. Pay Toll
2. Display Records
3. Generate Revenue Report
4. Delete Record
5. Exit
Choose an option: 2
--- Toll Collection Records ---
Vehicle Number: UK07ER4577
Vehicle Type: Light Commercial Vehicle
Toll Amount: 120.0
LCV type: Goods Vehicle

Vehicle Number: UP32EP0716
Vehicle Type: Light Motor Vehicle
Toll Amount: 50.0
Car Type: Hatchback

Total Revenue Collected: ₹170.0
```

```
≡ toll_records.txt
1 UK07ER4577,Light Commercial Vehicle,Goods Vehicle,120.0
2 UP32EP0716,Light Motor Vehicle,Hatchback,50.0
3 |
```

These two functions work in tandem to manage and present toll collection data effectively.

- **displayRecords()** is responsible for showing all toll transactions recorded in the system. It iterates through the stored `vehicleRecords` array, calls `displayInfo()` on each vehicle to print its number, type, and toll amount, and computes the cumulative toll revenue collected. At the end of the process, it prints the **Total Revenue Collected** in a clearly formatted manner. This function provides a complete overview of the toll booth's activity and financial performance.
- **writeRecordToFile(Vehicle vehicle)** is a utility method that ensures data persistence by appending each toll transaction to a file. It serializes the vehicle object into a comma-separated format using the `toFileString()` method and writes it to the file in append mode. This guarantees that all records are saved sequentially without overwriting existing data. Proper error handling ensures any file access issues are reported clearly.

Together, these functions enable both real-time display and long-term storage of toll data, supporting auditing, reporting, and monitoring of toll booth operations.

Function: loadRecordsFromFile()

```
private void loadRecordsFromFile() {
    try (BufferedReader br = new BufferedReader(new FileReader(FILE_NAME))) {
        String line;
        while ((line = br.readLine()) != null && count < vehicleRecords.length) {
            String[] parts = line.split(",");
            Vehicle vehicle = null;
            if (parts[1].equals("Light Motor Vehicle")) {
                vehicle = new LMV(parts[0], parts[2]);
            } else if (parts[1].equals("Light Commercial Vehicle")) {
                vehicle = new LCV(parts[0], parts[2]);
            } else if (parts[1].equals("Heavy Vehicle")) {
                vehicle = new HeavyVehicle(parts[0], parts[2]);
            }
            if (vehicle != null) {
                vehicleRecords[count++] = vehicle;
            }
        }
    } catch (IOException e) {
        System.out.println("No previous records found.");
    }
}
```

This function is designed to **load previously saved toll transaction data from a file** when the program starts, ensuring continuity between sessions.

- It opens the file using a `BufferedReader` and reads it line by line. Each line represents a single vehicle's toll record in a comma-separated format.
- The line is split into parts to extract relevant information such as the vehicle number, type, and subtype.
- Based on the `vehicleType` field, the appropriate subclass (`LMV`, `LCV`, or `HeavyVehicle`) is instantiated using the parsed data.
- The newly created `Vehicle` object is then stored into the `vehicleRecords` array, and the count is incremented accordingly.
- The method continues this process until all records are loaded or the array is full.
- If the file doesn't exist or an error occurs during reading, a fallback message "No previous records found." is displayed, indicating this may be the first time the system is running.

This function ensures **data persistence and recovery**, allowing the system to maintain historical toll data across multiple program executions.

Function: writeRevenueByVehicleType()

```
public void writeRevenueByVehicleType() {
    double LMVTotal = 0, LCVTotal = 0, HVTTotal = 0;

    try (FileWriter lmvwrite = new FileWriter("LMV_Revenue.txt"); FileWriter lcvwrite = new FileWriter("LCV_Revenue.txt"); FileWriter hvwrite = new FileWriter("HV_Revenue.txt")) {

        for (int i = 0; i < count; i++) {
            Vehicle v = vehicleRecords[i];
            double toll = v.calculateToll();

            if (v instanceof LMV) {
                LMVTotal += toll;
                lmvwrite.write(v.toFileString() + "\n");
            } else if (v instanceof LCV) {
                LCVTotal += toll;
                lcvwrite.write(v.toFileString() + "\n");
            } else if (v instanceof HeavyVehicle) {
                HVTTotal += toll;
                hvwrite.write(v.toFileString() + "\n");
            }
        }

        lmvwrite.write("Total LMV Revenue: ₹" + LMVTotal + "\n");
        lcvwrite.write("Total LCV Revenue: ₹" + LCVTotal + "\n");
        hvwrite.write("Total Heavy Vehicle Revenue: ₹" + HVTTotal + "\n");
    } catch (IOException e) {
        System.out.println("Error writing revenue Files: " + e.getMessage());
    }

    System.out.println("Revenue reports Generated Successfully !");
}
```

≡ LMV_Revenue.txt

```
1  UP32EP0716,Light Motor Vehicle,Hatchback,50.0
2  UK07DF4566,Light Motor Vehicle,SUV,70.0
3  BR01DR3422,Light Motor Vehicle,Hatchback,50.0
4  Total LMV Revenue: ₹170.0
5  |
```

```
≡ LCV_Revenue.txt
1 UK07ER4577,Light Commercial Vehicle,Goods Vehicle,120.0
2 UP32FG4533,Light Commercial Vehicle,Goods Vehicle,120.0
3 UP16JK5677,Light Commercial Vehicle,Minibus,100.0
4 Total LCV Revenue: ₹340.0
5
```

```
≡ HV_Revenue.txt
1 UK07QZ8900,Heavy Vehicle,Construction Vehicle,200.0
2 Total Heavy Vehicle Revenue: ₹200.0
3
```

This function is responsible for **generating separate revenue reports for each category of vehicle** (LMV, LCV, and Heavy Vehicle) and writing them into individual files.

- **Purpose:** To provide a clear breakdown of toll collection per vehicle category and enable easy auditing and analysis.
- **File Writers Used:**
 - LMV_Revenue.txt for Light Motor Vehicles
 - LCV_Revenue.txt for Light Commercial Vehicles
 - HV_Revenue.txt for Heavy Vehicles
- **How It Works:**
 - The method iterates through the entire vehicleRecords array.
 - For each vehicle, it checks its type using the instanceof operator.
 - Based on the vehicle type:
 - It adds the toll amount to the respective revenue total.
 - It writes the vehicle's details into the appropriate file using toString() format.
 - After processing all records, it appends the **total revenue collected** for each vehicle category at the end of each respective file.

- **Error Handling:** Any file-related exceptions are caught, and an appropriate message is displayed.
- **Final Output:** If successful, three neatly formatted revenue reports are created and a success message is displayed on the console.

This function demonstrates effective **file handling, revenue calculation, and data categorization**, making it a key part of the reporting feature in the Toll Tax Management System.

Main Method: TollSystem.java

```
----- Welcome to my Toll Tax Management System-----
1. Pay Toll
2. Display Records
3. Generate Revenue Report
4. Delete Record
5. Exit
Choose an option: 2
--- Toll Collection Records ---
Vehicle Number: UK07ER4577
Vehicle Type: Light Commercial Vehicle
Toll Amount: 120.0
LCV type: Goods Vehicle

Vehicle Number: UP32EP0716
Vehicle Type: Light Motor Vehicle
Toll Amount: 50.0
Car Type: Hatchback

Total Revenue Collected: ₹170.0

----- Welcome to my Toll Tax Management System-----
1. Pay Toll
2. Display Records
3. Generate Revenue Report
4. Delete Record
5. Exit
Choose an option: 3
Revenue reports Generated Successfully !
```

----- Welcome to my Toll Tax Management System-----

1. Pay Toll
2. Display Records
3. Generate Revenue Report
4. Delete Record
5. Exit

Choose an option: 1

Enter Vehicle Number: UK07DF4566

Select Vehicle Type: 1. LMV 2. LCV 3. Heavy Vehicle

1

Select LMV Type: 1. SUV 2. Sedan 3. Hatchback 4. Jeep

1

Processing Toll for Vehicle: UK07DF4566

Vehicle Number: UK07DF4566

Vehicle Type: Light Motor Vehicle

Toll Amount: 70.0

Car Type: SUV

Toll Paid Successfully!

Receipt Generated: Receipt_UK07DF4566.txt

----- Welcome to my Toll Tax Management System-----

1. Pay Toll
2. Display Records
3. Generate Revenue Report
4. Delete Record
5. Exit

Choose an option: 2

--- Toll Collection Records ---

Vehicle Number: UK07ER4577

Vehicle Type: Light Commercial Vehicle

Toll Amount: 120.0

LCV type: Goods Vehicle

Vehicle Number: UP32EP0716

Vehicle Type: Light Motor Vehicle

Toll Amount: 50.0

Car Type: Hatchback

Vehicle Number: UP32FG4533

Vehicle Type: Light Commercial Vehicle

Toll Amount: 120.0

LCV type: Goods Vehicle

Vehicle Number: UK07DF4566

Vehicle Type: Light Motor Vehicle

Toll Amount: 70.0

Car Type: SUV

Vehicle Number: BR01DR3422

Vehicle Type: Light Motor Vehicle

Toll Amount: 50.0

Car Type: Hatchback

Vehicle Number: UP16JK5677

Vehicle Type: Light Commercial Vehicle

Toll Amount: 100.0

LCV type: Minibus

Vehicle Number: UK07QZ8900

Vehicle Type: Heavy Vehicle

Toll Amount: 200.0

Heavy Vehicle Type: Construction Vehicle

Total Revenue Collected: ?710.0

This is the **entry point** and **main system controller** for the Toll Tax Management System. It provides a **menu-driven interface** allowing users to interact with the toll booth functions via the console.

Main Features:

- A continuous while(true) loop runs until the user chooses to exit.
- The user can:
 1. **Pay Toll** – Enter vehicle details, select type and subtype, and proceed with toll collection.
 2. **Display Records** – View details of all vehicles and total revenue collected.
 3. **Generate Revenue Report** – Generate files based on vehicle type showing toll amounts collected.
 4. **Delete Record** – Delete a vehicle entry based on its vehicle number.
 5. **Exit** – Exit the system safely.

Highlights:

- **User Interaction:** Uses Scanner for input and employs input validation using try-catch to avoid InputMismatchException.
- **Vehicle Creation:** Based on user input, specific Vehicle subclass (LMV, LCV, or HeavyVehicle) is instantiated with subtype.
- **Toll Collection Integration:** The selected vehicle is processed through the collectToll() method of the TollBooth class, where the record is saved, and a receipt is generated.
- **Extensibility:** The structure supports easy expansion in the future—for example, integrating a GUI or adding more vehicle types.

Error Handling:

- If the user enters an invalid option or a non-integer, an appropriate message is shown.
- This keeps the application from crashing and ensures a smooth user experience.

